

Ben-Gurion University of the Negev
The Faculty of Engineering Sciences
Department of Software and Information Systems Engineering

Playlist and Transition Optimization for Parties

*A Computational Model for Finding a Path Between Songs Based on Energy
Progression and Smooth Transitions*

Final Project Report

Ohad Motola (209232420) · Shelly Kritsberg (323866285)
Yuval Antman (212054167) · Tomer Filo (206969750)

Github : [playlist organizer](#)

2025–2026

Table of Contents

Table of Contents	2
1.1 The Engineering and Research Problem	3
1.2 Proposed Computational Approach	3
1.3 Computational Tools Considered	4
1.4 Success Metrics.....	4
1.5 Initial Sources	5
1.6 Project Timeline.....	5
Chapter 2: Background and Literature Review	6
2.1 Gap Analysis and Proposed Solution.....	6
2.2 In-Depth Understanding of Existing Methods and Tools	6
2.3 Intelligent Comparison: Evolution and Differences Between Methods	8
2.4 Comparing Existing Solutions to Our Proposed Product	8
2.5 Relevant Papers, Tools, and Resources.....	9
Chapter 3: Computational Methods and Evaluation Methodology.....	10
3.1 System Objectives, Architecture, and Information Flow	10
3.2 Module-Level Design	11
3.3 The Method Family and Ablation Design	12
3.4 Relationship to Methods in the Background Chapter.....	14
3.5 Evaluation Methodology and Success Criteria.....	15
Chapter 4: Experimental Results and Discussion	16
4.1 Dataset and Feature Engineering.....	16
4.2 Methods Compared	16
4.3 Overall Results (averaged across all 360 playlists).....	16
4.4 Discussion of Results	16
4.5 Current System Architecture Summary	19
4.6 Limitations and Future Work	19
4.7 Conclusions	20
Appendices.....	21
Appendix A - BIBS Pseudocode	21
Appendix B - Metric Definitions	21
Appendix C - Additional Evaluation Formulas	22

Appendix D - results.....	22
Appendix E - Works Cited.....	26

Chapter 1: Introduction and project Proposal

1.1 The Engineering and Research Problem

Traditional music recommendation systems usually focus on matching individual songs according to semantic similarity, collaborative filtering, or content-based filtering, but they do not sufficiently take into account the temporal context of listening and the quality of the transition from one song to the next. As a result, even when each song is suitable on its own, the entire sequence may sound fragmented, random, and unnatural, especially in the context of parties and events where a continuous flow of energy is required.

The gap that the project seeks to fill is the development of a computational model that formulates playlist creation as a problem of finding an optimal path between songs, so that the sequence is planned in advance according to energy progression, acoustic and harmonic compatibility, and smoother transitions between tracks.

Accordingly, the main objectives of the project are:

- To represent each song using a collection of relevant acoustic features such as tempo, energy, and musical key;
- To build a cost function that measures the quality of the transition between songs;
- To develop an optimization mechanism that creates playlists according to a desired energy trajectory.

In addition, the project aims to improve not only the selection of the song order but also the quality of the connection between them in practice, so that the final product will be a coherent, flowing musical sequence that is more suitable for a party environment.

The idea for the project arose from exposure to student party culture, in which a sense of fragmentation between songs is often noticeable. From this, an interest arose in examining how a higher-quality musical sequence can be generated, one that simulates and even improves the work of a DJ.

1.2 Proposed Computational Approach

The computational approach proposed in the project is to treat the problem of playlist creation not as a local choice of 'the next most similar song,' but as a serial optimization problem of an entire sequence. For this purpose, each song is represented as a vector of acoustic features, which includes characteristics such as danceability, valence, key, energy, tempo, and loudness, as well as harmonic compatibility according to the Camelot Wheel - a circular representation of musical scales that enables the identification of smooth harmonic transitions between songs.

This representation makes it possible to translate the songs into a computational space in which differences between songs can be measured not only according to general similarity, but also according to their suitability for a continuous transition within a musical set.

Based on this representation, the song catalog is modelled as a graph: each song is a node, and every possible transition between two songs is an edge with a cost. This cost reflects the acoustic and harmonic gap between the two songs. The problem is thus formulated as a search for an optimal path in a graph.

1.3 Computational Tools Considered

For the purpose of finding an optimal path between songs, the project examines several central computational methods on the musical graph:

- Greedy algorithms - at each stage choose the transition with the lowest cost, providing a simple and fast baseline for creating an initial sequence;
- Constraint satisfaction programming - defines not only a cost function but also structural conditions on the playlist (e.g., gradual energy increase, harmonic compatibility);
- Bidirectional Beam Search with recursive decomposition - the primary proposed method; expands partial paths both from the playlist start and from the playlist end simultaneously, meeting at midpoints, with bottleneck-guided anchor selection;
- Reinforcement learning - considered but rejected for offline sequencing (see Chapter 3).

It should be noted that the problem is similar in nature to the Traveling Salesman Problem, since the number of possible sequences grows very quickly as the number of songs increases. The goal of the project is therefore not necessarily to find a single globally optimal solution, but to find good and practical sequences within a reasonable time, balancing musical quality and computational efficiency.

1.4 Success Metrics

The success of the system is examined using metrics that evaluate the quality of the musical sequence as a whole:

- Total trajectory cost (RMSE between target Energy-Valence trajectory and actual sequence EV values) - lower is better;
- Transition smoothness (sum of C_{trans} across consecutive pairs) - lower is better;
- Global coherence (ratio of sequential variance to population variance) - higher is better;
- Anchor alignment cost (BIBS-specific: structural quality of midpoint selections);
- Human satisfaction evaluation - listener ratings on a five-point Likert scale (planned; not carried out - see Section 3.5).

The final evaluation additionally reports a composite ranking that averages each method's rank across ten computational metrics (arc RMSE, total / average transition cost, p90 / p95 / max transition, DTW shape, ratio-adherence RMSE, method time, and inverted global coherence). This composite is the headline metric used in Chapter 4.

1.5 Initial Sources

- Bittner et al. (2017): Automatic Playlist Sequencing and Transitions. ISMIR.
- Liebman et al. (2015): DJ-MC - A Reinforcement-Learning Agent for Music Playlist Recommendation. arXiv.
- Bonnin & Jannach (2014): Automated Generation of Music Playlists: Survey and Experiments.
- Flexer et al. (2008): Playlist Generation Using Start and End Songs.
- Schweiger et al. (2025): The Impact of Playlist Characteristics on Coherence in User-Curated Music Playlists.
- Zehren et al. (2022): Automatic Detection of Cue Points for the Emulation of DJ Mixing.
- Vande Veire & De Bie (2018): From Raw Audio to a Seamless Mix: Creating an Automated DJ System for Drum and Bass.
- Tomasi et al. (2023): Automatic Music Playlist Generation via Simulation-based Reinforcement Learning. Spotify.

1.6 Project Timeline

- April 12 – 19: Collecting and organizing data
- April 20 – May 3: Creating computational representations and cost functions
- May 4 – 17: Implementing optimization methods
- May 18 – 31: Experiments and comparison between methods
- June 1 – 14: Analyzing results and drawing conclusions
- June 15 – 24: Writing, editing, and presentation

Chapter 2: Background and Literature Review

Optimizing playlist flow and sequencing: a model to find the best path across a set of songs while maintaining a predefined ‘energetic trajectory’ and ensuring high-quality local transitions.

2.1 Gap Analysis and Proposed Solution

Most traditional music recommendation systems (like Collaborative Filtering or Content-Based Filtering) focus on recommending individual tracks based on semantic similarity or past user interactions, often ignoring the temporal context and the holistic listening experience. While automated DJ systems and playlist generators have been explored in the past, they typically suffer from fundamental limitations: they either focus strictly on local transition mechanics without a long-term structural goal, or they rely on generic Reinforcement Learning (RL) aiming to minimize track skips. As Spotify researchers (Tomasi et al. 2023) note, conventional techniques frequently suffer from a “misalignment between offline model objectives and online user satisfaction metrics,” failing to guarantee a cohesive musical narrative.

Our project introduces a significant innovation by formulating playlist generation not as a static, computationally heavy graph-solving problem, but as a dynamic, recursive sequential selection process guided by bidirectional beam search with lookahead. The system computes real-time transition costs, incorporating acoustic and harmonic disparities (using the Camelot Wheel), relative to a predefined global Energy-Valence trajectory. When stochastic mode is enabled, the system optionally samples from the Top-K anchor candidates at each anchor selection step instead of always selecting the single best candidate. This introduces controlled randomness (novelty and diversity) while bidirectional lookahead planning preserves global coherence.

2.2 In-Depth Understanding of Existing Methods and Tools

2.2.1 Similarity-Based and Sequence Models

Early research primarily relied on audio similarity and metadata to generate playlists. For example, Bonnin et al. (2014) [2] mentions the use of GPS location of users as metadata on page 8 in the table of additional information used. Flexer et al. (2008) [3] demonstrated the generation of playlists by defining a start and end song, using spectral similarity (specifically, Kullback-Leibler divergence between Mel Frequency Cepstrum Coefficients, or MFCCs) to create a smooth, interpolating transition between the two anchor points. However, relying solely on similarity often results in overly homogeneous playlists that lack the dynamic variation required for an engaging listening experience (Bonnin et al. (2014) [2] page 8 under section 4.1); also, they discuss generating a playlist while we discuss the situation where a playlist is given and we want to find fitting energetic sequences in it. Moreover, Flexer et al. [3] (on page 177 under Table 8) note that finding a proper transition can fail if the database lacks tracks that fit logically between two highly contrasting anchor songs.

2.2.2 Graph Theory and Combinatorial Optimization

A more sophisticated approach models track sequencing as a graph traversal problem. Bittner et al. (2017) [4] (on page 3 under section 3.1.2) framed the optimal ordering of a playlist as finding the Shortest Hamiltonian Path in a complete weighted graph. In their model, edge weights are determined by Euclidean distances in a feature space that includes tempo (mapped logarithmically), key (mapped via the circle of fifths), and timbre (in section 3.1.1 in Bittner et al.). Because finding an exact Hamiltonian path is NP-complete, greedy algorithms (e.g., HAM-1, HAM-2) and Traveling Salesman Problem (TSP) approximations are widely used to minimize overall transition costs. By combining this macro-level sequencing with micro-level cue point selection at major structural boundaries, they demonstrated that beat-aligned transitions strongly enhance listener satisfaction and mask otherwise abrupt changes (under section 4, Transitions).

However, a major vulnerability in myopic greedy selection is what is known as the “Greedy Trap” (or “leftovers” problem). Bittner et al. explicitly documented this limitation, noting that “an undesirable artifact is the presence of poor track pairings at the tail of the sequencing for HAM-1” [4]. This highlights the strict necessity for algorithms that can look ahead rather than just taking the immediate lowest cost.

2.2.3 Reinforcement Learning and Markov Decision Processes

RL has been utilized to adapt to listener preferences dynamically. Liebman et al. (2015) [5] (on page 591 in their Abstract and part 2 Reinforcement learning framework) introduced DJ-MC, an agent that models playlist recommendation as an episodic MDP, learning user preferences over both individual songs and song-to-song transitions by representing songs through rich spectral and rhythmic descriptors (under section 3, Modeling). Recently, Spotify developed an Action-Head Deep Q-Network (AH-DQN) trained in a simulated environment to optimize user satisfaction metrics (like completion rates) by sequentially selecting tracks [1]. These models highlight the absolute importance of sequential awareness, proving that the pleasure derived from a song is directly affected by its relative position in a sequence.

While deep RL can be complex to train, planning algorithms derived from this domain (such as Beam Search or Monte Carlo Tree Search) provide powerful “RL-lite” alternatives (Liebman et al. under section 5, DJ-MC). They evaluate partial sequences and look several steps ahead to avoid dead ends, proving highly effective for sequential decision making.

2.2.4 DJ Automation and Intra-Track Analysis

Creating a seamless mix requires analyzing the internal structure of songs to find “switch points” or cue points. Zehren et al. (2022) [6] (under sections Rule-Based Approach and Novelty Detection in pages 70–76) used expert DJ rules and statistical novelty detection algorithms (like Foote’s checkerboard kernel) to identify structural boundaries, such as downbeats at the start of a musical period, suitable for crossfading. Vande Veire and De Bie (2018) [7] built a complete open-source auto-DJ system for Drum and Bass that automates beat-matching, downbeat tracking, and applies specific crossfade profiles based on structural segmentation.

Audio-frame transition cost (timbre, chroma, loudness over a beat-aligned overlap window, as in Bittner et al. Algorithm 1) requires audio and is out of scope for the current project. Our `C_trans` is the faithful feature-level adaptation. Audio-level crossfade optimization is left as future work.

2.3 Intelligent Comparison: Evolution and Differences Between Methods

2.3.1 Graph/Optimization vs. Reinforcement Learning

Graph-based approaches (like TSP or Constraint Satisfaction Programming) are highly explainable and allow developers to enforce strict musical rules, such as preventing dissonant harmonic clashes or mandating a specific tempo trajectory [4]. They also naturally support the incorporation of explicit mathematical constraints, which is highly beneficial when crafting a deliberate musical journey. However, calculating the cost matrix for large catalogs is computationally expensive, $O(N^2)$ (Bittner et al. page 3 section 3.1.2). In contrast, RL models (like DJ-MC [5]) excel at personalizing content dynamically based on user feedback (skips/listens) [1][5]. Yet, RL models often act myopically and struggle to enforce strict long-term structural narratives, such as the classic “Hero’s Journey” or “Double Peak” energy trajectories used by professional DJs to prevent listener ear fatigue.

By adopting a dynamic pipeline, our system calculates cost functions in real-time, allowing the track pool to be updated on the fly while following the “Energy Trajectory” and having the start and end songs as context. This mirrors a real live DJ environment, where a DJ responds to the crowd’s energy sequentially rather than having the entire night perfectly mapped out in advance.

2.3.2 Inter-track vs. Intra-track Optimization

While traditional Recommender Systems focus entirely on inter-track mixability (which song should play next), automated DJ research highlights the absolute necessity of intra-track mixability (where exactly the crossfade should occur). Zehren et al. [6] note that switch points profoundly impact the continuous listening experience. Studies show that misaligned beats or transitioning during vocals are primary causes of perceived poor quality in automated mixes, alongside incompatible timbral shifts (Bittner et al. (2017) [4], pages 5–6).

2.3.3 Optimization vs. Diversity

Greedy algorithms maximize mathematical similarity but destroy musical novelty. By incorporating a probabilistic sampler that draws from the Top-K lowest-cost candidates, developers can balance optimization with critical musical diversity, preventing the algorithm from getting stuck in repetitive loops. This is exactly the role of the stochastic pruning added to our best-performing methods (Chapter 3).

2.4 Comparing Existing Solutions to Our Proposed Product

Existing commercial and academic systems largely fail to integrate the macro-level planning of a DJ set with the micro-level execution of smooth audio transitions. For example, Spotify’s default shuffle or basic recommendation algorithms can create jarring, disjointed experiences because they evaluate tracks independently. While advanced RL models aim to maximize listen time [1], they do not explicitly plan an “Energy Trajectory” (e.g., Ramp-Up for a warmup set, or Rollercoaster for a peak-time party).

Our proposed product tackles this directly through a Bottleneck-Guided Recursive Bidirectional Beam Search (BIBS) and the meet-in-the-middle beam family derived from it. At each stage, the algorithm evaluates candidate tracks according to their trajectory cost, harmonic compatibility, and transition quality relative to the current playlist boundaries. To avoid the “Greedy Trap”

observed in previous literature [4], the system performs bidirectional beam search with lookahead before committing, dividing the playlist into smaller subproblems. When stochastic mode is enabled, the algorithm optionally samples from the Top-K ranked candidates instead of always selecting the single best, introducing controlled diversity while preserving trajectory adherence and transition quality.

We measure the success of this model using two primary metric families: (1) convergence to the energy trajectory (RMSE between the target curve and the generated sequence), and (2) sequential coherence. As Schweiger et al. (2025) [8] highlight, traditional coherence metrics are flawed; we therefore apply their formal definition for measuring playlist coherence “based on the sequential ordering of tracks,” mathematically evaluating the order-dependent smoothness of BPM and harmonic transitions over time.

2.5 Relevant Papers, Tools, and Resources

2.5.1 Audio Feature Extraction Tools (Future Work)

- Librosa: Python library for Music Information Retrieval. Excellent for extracting Mel-spectrograms, MFCCs, beat tracking, and onset detection.
- Essentia: highly optimized C++ library with Python bindings; recommended over Librosa for processing large datasets quickly.

2.5.2 Dataset Used in This Project

- Spotify Tracks Dataset (spotify_tracks.csv): pre-computed acoustic features (tempo, energy, valence, danceability, loudness, key, mode, time_signature, speechiness, acousticness, instrumentalness, liveness) for a curated pool of tracks. Used as the primary experimental dataset.
- FMA (Free Music Archive) & MTG-Jamendo: open datasets providing hundreds of thousands of tracks with pre-computed acoustic features, tags, and open audio files suitable for rendering crossfades.
- Million Playlist Dataset (MPD): useful for analyzing human curation patterns and sequence coherence; also used in Schweiger et al. (2025) [8].
- UnmixDB: ground-truth annotations of DJ mixes, cue points, and beat tracking, essential for future automatic crossfading and switch-point detection; also used in Zehren et al. (2022) [6].

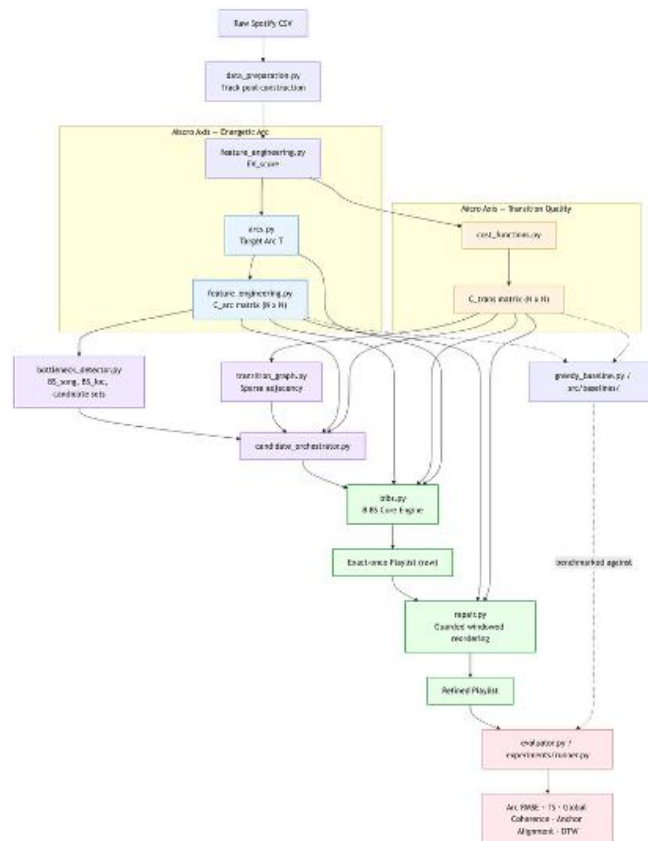
The full list of works cited is provided in Appendix E.

Chapter 3: Computational Methods and Evaluation Methodology

3.1 System Objectives, Architecture, and Information Flow

The system constructs a complete playlist from a fixed, predefined set of tracks, where every track must be placed exactly once. The goal is to find a feasible, low-cost ordering that follows a predefined energy and mood trajectory as closely as possible, while also reducing the transition cost between consecutive tracks. This creates two requirements: at the macro level, the sequence should match the intended energy and valence development across playlist positions; at the micro level, transitions between an outgoing and an incoming track should be musically coherent, mainly in tempo and harmonic compatibility. Since the track set is closed, the system must also address the “leftovers” problem: difficult-to-place tracks cannot be ignored, and if left until the end they may damage the quality of the final sequence.

The architecture is a hybrid process combining pre-computation, bottleneck identification, recursive bidirectional search, and local micro-optimization. Input consists of the track pool, the desired playlist trajectory, and the musical features of each track (energy, valence, BPM, key). The first stage evaluates the fit between every track and every position via the trajectory cost. Based on this, the system identifies bottleneck tracks - tracks that are hard to place because even their best position still produces a relatively high distance from the target trajectory. In parallel, the system computes transition costs between candidate pairs. It then applies a bottleneck-



guided recursive bidirectional search, and finally a base-case local matching phase for small remaining gaps ($\leq \text{base_case_size}$, default 4).

Figure 3.1 - System architecture / information flow: from raw Spotify CSV through feature engineering, arc and cost computation, bottleneck detection, candidate orchestration, the BIBS core engine, repair, and evaluation.

3.2 Module-Level Design

3.2.1 Formal Problem Definition

Inputs: a collection of N tracks, each with identifiers (id, name, artist) and a suite of 13 musical features including duration, energy, valence, and tempo (BPM); a designated starting track s_{start} and ending track s_{end} ; a playlist length L ($= N$, as every track must appear exactly once); and global parameters used to synthesize the target Energy-Valence trajectory. Output: a complete ordered sequence P of length L that optimizes track-to-track fluidity while adhering to the specified emotional narrative.

3.2.2 Global Narrative: Energy-Valence Trajectory Cost

The Energy-Valence trajectory is the blueprint for the intended emotional progression across the playlist. The system derives a composite EV_score for each track via a normalized weighted sum of its primary features:

$$\text{EV_Score}(s) = (\text{wE} \cdot \text{norm_energy} + \text{wV} \cdot \text{norm_valence} + \text{wD} \cdot \text{norm_danceability} + \text{wL} \cdot \text{norm_loudness} + \text{wT} \cdot \text{norm_tempo}) / (\text{wE} + \text{wV} + \text{wD} + \text{wL} + \text{wT})$$

In the implemented system, all weights are equal ($\text{wE} = \text{wV} = \text{wD} = \text{wL} = \text{wT} = 1.0$). Each feature is min-max normalized to $[0, 1]$ across the track pool before the weighted sum. Tempo thus appears in both EV_score (normalized) and in the rhythm component of C_trans (log-ratio); this is intentional and retained as the official definition.

The target progression is defined over the playlist duration as:

$$\begin{aligned} \text{A_EV}(t) &= \text{EV_Score_Start} + \text{progress}(t) \cdot (\text{EV_Score_End} - \text{EV_Score_Start}), \\ \text{progress}(t) &= (t - 1) / (L - 1) \end{aligned}$$

The trajectory cost, measuring the macro-level distance between a track and its target position, is $\text{C_trajectory}(s, t) = |\text{EV_Score}(s) - \text{A_EV}(t)|$. Lower values signify better alignment with the intended global narrative.

Non-linear arc shapes (double_peak, log_rise, inverted_parabola, wave) are implemented in `src/arcs.py` and used in experiments. The linear arc remains the default; the final suite evaluates all five shapes (see Chapter 4).

3.2.3 Bottleneck Identification and Search Filtering

To mitigate the leftovers problem, this module identifies tracks and positions that are musically restrictive, ensuring these “bottlenecks” are prioritized early.

Song Bottleneck Score: $\text{BS_song}(s) = \text{mean of the 5 lowest values in } \{ \text{C_trajectory}(s, t) \mid t = 1..L \}$. High scores indicate tracks that are globally difficult to fit and require early placement.

Location Bottleneck Score: $\text{BS_loc}(t) = \text{mean of the 2 lowest values in } \{ \text{C_trajectory}(s, t) \mid s = 1..N \}$. This identifies positions with few suitable candidates.

3.2.4 Local Fluidity: Transition Cost

The transition cost C_{trans} between consecutive tracks decomposes into three musical dimensions: $C_{\text{trans}}(u, v) = \alpha \cdot d_{\text{rhythm}} + \beta \cdot d_{\text{tonality}} + \gamma \cdot d_{\text{texture}}$, with default weights $\alpha = 1.0$ (rhythm), $\beta = 0.4$ (tonality), $\gamma = 0.6$ (texture).

- **Rhythmic distance:** $d_{\text{tempo}} = |\log_2(\text{tempo}_v / \text{tempo}_u)|$; $d_{\text{rhythm}} = d_{\text{tempo}} + \text{meter_penalty}$ (0.25) if time signatures differ. The log-ratio captures perceptual doubling (120→60 BPM = distance 1.0, same as 60→30).
- **Harmonic distance:** Camelot sector + mode distance (primary): $\text{sector_dist} = \min(|\Delta\text{sector}|, 12 - |\Delta\text{sector}|)$; $\text{mode_dist} = 0$ if modes match else 1; $d_{\text{tonality}} = \text{sector_dist} + \text{mode_dist}$.
- **Textural distance:** $d_{\text{texture}} = \text{L2_norm}(\text{texture}_v - \text{texture}_u)$ over danceability, energy, norm_loudness, speechiness, acousticness, instrumentalness, liveness, valence - all normalized to [0,1].

3.2.5 Core Engine: Recursive Bidirectional Search and Anchor Locking

This central module recursively subdivides playlist intervals, expanding inward from established boundaries toward a central anchor. Given interval $[\text{left_pos}, \text{right_pos}]$, let $M = \text{floor}((\text{left_pos} + \text{right_pos})/2)$. The system simultaneously expands a forward beam from $\text{left_pos}+1$ toward $M-1$ (seeded by the left boundary track) and a backward beam from $\text{right_pos}-1$ toward $M+1$ (seeded by the right boundary track). Each beam keeps beam_width partial paths, pruned by a composite step score:

```
StepScore(x, prev, pos) = trajectory_weight · norm(C_trajectory[x, pos]) +
trans_weight · norm(C_trans[prev, x]) - btk_weight · norm(BS_song[x])
```

The negative bottleneck term prioritizes tracks that are harder to place. After the beams complete, an anchor track is selected at M from the joint forward/backward beam endpoints plus a bottleneck-prioritized candidate set, with a balance penalty preventing uneven left/right transition costs. Once the anchor is locked, the interval bifurcates into $[\text{left_pos}, M]$ and $[M, \text{right_pos}]$ and the process repeats recursively.

3.2.6 Final Resolution: Base-Case Local Matching

When a segment has $\leq \text{base_case_size}$ (default 4) empty slots, the system evaluates all valid permutations of a candidate set (up to $\text{base_case_max_candidates} = 12$ per segment). Scoring shifts weight strongly toward transition smoothness: $\text{BaseScore} = \text{base_trajectory_weight} \cdot \sum \text{norm}(C_{\text{trajectory}}) + \text{base_trans_weight} \cdot \sum \text{norm}(C_{\text{trans}}) + \text{btk_weight} \cdot \sum \text{norm}(BS_{\text{song}})$, with $\text{base_trajectory_weight} = 0.3$ and $\text{base_trans_weight} = 1.5$ - mimicking a DJ who matches beats at the moment of crossfade.

The full BIBS pseudocode is provided in Appendix A.

3.3 The Method Family and Ablation Design

All methods share the same cost functions, trajectory definition, and endpoint selection (the ev_proximity mode: start/end tracks are the songs whose EV score is closest to the natural low/high endpoints of the trajectory). They differ only in how the interior ordering is produced.

The ablation was designed around a single question: which components of BIBS actually drive quality? We therefore started from BIBS, isolated its weakness (recursive midpoint anchoring hurts transitions), and then transplanted its best components - candidate orchestration and stochastic pruning - onto the simpler meet-in-the-middle (MM) beam, one at a time.

3.3.1 Baselines

Baseline	Description
random	Random permutation of tracks. Floor reference - no optimization.
transition_greedy	At each step picks the track with the lowest transition cost from the previous. Ignores trajectory. Best transition metric but worst arc adherence.
arc_assignment	Hungarian assignment - globally optimal track-to-position mapping by arc cost. Zero transition awareness. Theoretical arc-RMSE lower bound.
flexer_interp	Assigns tracks to positions using acoustic interpolation ratios between start and end. No trajectory, no transitions.
forward_beam	Left-to-right beam search scoring $\text{arc_weight} \cdot C_{\text{arc}} + \text{trans_weight} \cdot C_{\text{trans}}$. Deterministic, single-direction.
mm_beam	Bidirectional meet-in-the-middle beam. Forward beam covers $0 \rightarrow \text{mid}$, backward beam covers $N-1 \rightarrow \text{mid}$, committed at midpoint. Non-recursive, deterministic.
greedy_arc_trans	Greedy per-step minimizing $\text{arc_weight} \cdot C_{\text{arc}} + \text{trans_weight} \cdot C_{\text{trans}}$. Direct baseline for BIBS.

3.3.2 Our Methods (ablation)

Method	What was added vs. the plain MM beam / BIBS baseline
bibs_current	Recursive bidirectional beam with bottleneck-guided candidate orchestration and midpoint anchor placement; recurses until base-case intervals are brute-forced.
bibs_repair	BIBS output passed through the guarded local repair pass (below).
bibs_stochastic	BIBS with stochastic beam pruning: samples from top-K candidates with probability $\propto \exp(-(\text{score} - \text{min}) / \text{temperature})$. Different seed $\rightarrow$ different valid ordering.
bibs_stoch_repair	Stochastic BIBS + repair.
bibs_parallel	Level-synchronous BIBS: all intervals at the same recursion depth place anchors simultaneously, with conflict arbitration.
mm_beam_stochastic	Plain MM beam with stochastic beam pruning (same softmax top-K sampler). Non-recursive.
mm_beam_stochastic_repair	Stochastic MM beam + repair.
mm_beam_orch_stochastic	Stochastic MM beam WITH BIBS-style candidate orchestration: candidates pre-filtered by arc cost, transition-graph neighbors, and bottleneck priority instead of iterating all tracks.
mm_beam_orch_stochastic_repair	Orchestrated stochastic MM beam + repair.
bibs_tempo_aware	BIBS with a soft BPM-jump penalty (waived when a track is an excellent trajectory fit). Implementation had runtime issues - see caveats.
bibs_alpha2	BIBS with rhythm weight $\alpha=2.0$ in C_{trans} (double tempo weight).
mm_beam_alpha2	MM beam with $\alpha=2.0$.
bibs_traj_first	BIBS with trajectory-first endpoint selection: trajectory defined from the pool's natural EV range first, then start/end chosen to fit positions 0 and N-1.

3.3.3 Three Reusable Components

The repair pass.

After any algorithm produces an ordering, the repair pass finds the `max_windows` transitions with the highest cost, extracts a small window of $\pm \text{window_radius}$ tracks around each, tries all permutations, and accepts a reorder only if total window transition cost decreases AND arc RMSE does not increase beyond `arc_guard`. Endpoints are always fixed. It is a local micro-optimizer that patches the worst seams; it reliably reduces p95 and max transition with minimal arc cost, runs in $\sim 0.13\text{--}0.17$ s on top of any method, and never worsens arc adherence beyond the guard.

Stochastic pruning.

In a standard beam search only the top-K scoring beams survive (deterministic argmin). In the stochastic variant, survival uses softmax sampling: restrict candidates to top-k by score, then sample `beam_width` of them without replacement with probability $\propto \exp(-(\text{score} - \text{min})/\text{temperature})$. Better candidates are more likely to survive, but selection is not deterministic - different `random_seed` yields a different but equally valid ordering. In a real app, users would run the algorithm multiple times and get varied playlists that still respect quality constraints, rather than the same static ordering.

Candidate orchestration.

In plain MM beam, each expansion step considers all remaining tracks ($O(N)$ per step). The orchestrated variant pre-filters candidates with the same `CandidateOrchestrator` as BIBS: an arc-cost filter (only tracks whose `C_arc[track, position]` is below a threshold), transition-graph neighbors (prefer tracks connected to the previous track in the learned graph), and bottleneck priority (boost hard-to-place tracks so they are not stranded). This shrinks the effective candidate pool without losing quality - the main efficiency gain in BIBS - and in MM beam additionally biases expansion toward musically coherent sequences.

3.4 Relationship to Methods in the Background Chapter

3.4.1 Methods We Chose Not to Use as Primary Approach

Greedy algorithms were rejected as a primary method due to the leftovers problem, but serve as primary baselines (`transition_greedy`, `greedy_arc_trans`). Constraint Satisfaction Programming was not adopted because hard, pre-defined constraints do not fit our problem well; our approach is effectively a soft CSP with a continuous, phase-dependent cost function. Reinforcement Learning (DJ-MC, AH-DQN) was rejected because RL targets interactive environments with continuous user feedback, whereas our task is offline sequencing of a static pool, and the full-sequence MDP state space is intractable here.

3.4.2 Methods We Use and How We Apply Them

Transition cost (Bittner et al., 2017): the DJ-transition feature space is adopted as our micro-level module, with a key departure - weights shift by phase (trajectory dominant during macro recursion, transition dominant during base-case resolution). Meet-in-the-middle (Holte et al., 2016): the structural backbone; we apply MM locally and recursively, and adapt it to a combinatorial sequencing space with item-uniqueness constraints, prioritizing heuristic fluidity over provably shortest paths. Beam search is embedded within every recursion level. Top-K stochastic mode injects controlled diversity via softmax sampling over candidate scores.

3.4.3 Primary Innovation

The central contribution is the integration of bidirectional search, beam search, recursive decomposition, and bottleneck prioritization into a novel architecture for closed-catalog sequencing with a predefined energy trajectory and fixed start/end tracks - together with the finding (Chapter 4) that BIBS's reusable components (bottleneck-aware candidate orchestration and stochastic pruning) are best deployed on a committed MM beam rather than inside recursion.

3.5 Evaluation Methodology and Success Criteria

Trajectory adherence (RMSE): $RMSE = \sqrt{(1/L) \cdot \sum_t C_trajectory(P[t], t)^2}$. Transition smoothness: $TS = \sum C_trans(P[i], P[i+1])$; we also report average and p95 transition cost. Global coherence: $COH = 1 - (\text{mean_seq_sq} / \text{mean_pop_sq})$, where mean_seq_sq is the mean squared sequential transition cost and mean_pop_sq is the mean over all off-diagonal pool pairs; COH near 0 = no smoother than random, near 1 = near the achievable lower bound. Auxiliary metrics: DTW shape distance, transition percentiles (p90/p95/max), playlist diversity across stochastic runs, and ratio-adherence RMSE (Flexer-inspired). Full metric definitions are in Appendix C.

A human satisfaction evaluation (blind listener ratings of playlists on a five-point Likert scale) was planned but could not be carried out within the project timeframe due to time constraints. It is left as future work (see Section 4.6.2).

3.5.1 Experimental Setup

The final evaluation is a full suite: 3 experiment types $\times$ 3 playlist lengths ($N = 60 / 90 / 120$), 40 playlists per cell (360 playlists total), 20 algorithms per playlist, across 5 trajectory shapes (linear, log_rise, wave, double_peak, inverted_parabola) and 3 genre-diversity settings (single_genre, similar_genres, all_genres). Methods are compared under identical conditions per playlist (same pool, same start/end pair, same cost matrices). For stochastic methods, metrics are averaged across independent runs.

Chapter 4: Experimental Results and Discussion

4.1 Dataset and Feature Engineering

All experiments use the Spotify Tracks dataset (`data/spotify_tracks.csv`). Each track has 13 acoustic features: tempo, energy, valence, danceability, loudness, speechiness, acousticness, instrumentalness, liveness, key, mode, time_signature, and duration.

EV_score combines energy, valence, danceability, normalized loudness, and normalized tempo with equal weights (each min-max normalized to $[0,1]$ across the pool). A Camelot key representation (sector 1–12, mode A/B) is derived from Spotify's key and mode fields. Texture vectors for `d_texture` use danceability, energy, `norm_loudness`, speechiness, acousticness, instrumentalness, liveness, and valence. Start and end tracks are selected automatically as the EV-closest songs to the natural trajectory endpoints, subject to a minimum EV gap.

4.2 Methods Compared

Twenty configurations are evaluated: seven baselines (`random`, `transition_greedy`, `arc_assignment/Hungarian`, `flexer_interp`, `forward_beam`, `mm_beam`, `greedy_arc_trans`) and thirteen BIBS- and MM-beam-based variants - including `repair`, `stochastic`, `orchestration`, `alpha2`, `tempo-aware`, `parallel`, and `trajectory-first` configurations. Full descriptions are in Section 3.3.

4.3 Overall Results (averaged across all 360 playlists)

Composite score = average rank across ten metrics: `arc_rmse`, `total_transition_cost`, `average_transition_cost`, `p90_transition`, `p95_transition`, `max_transition`, `dtw_shape`, `ratio_adherence_rmse`, `method_time_s`, and `global_coherence` (inverted). Lower is better.

The ranking shows that the top four methods are all MM-beam stochastic variants. The best overall method is `mm_beam_orch_stochastic`, while `mm_beam_orch_stochastic_repair` is the strongest practical configuration because it achieves the best worst-case transition behavior. The result suggests that the best final architecture is not recursive BIBS alone, but MM Beam enhanced with stochastic pruning, BIBS-style candidate orchestration, and repair. This ranking is visualized in Fig. A1, and the main trajectory-transition trade-off is shown in Fig. A2.

4.4 Discussion of Results

4.4.1 Our best methods win

The top four methods by composite ranking are all ours: the stochastic and orchestrated MM-beam variants. Two stand out in particular:

- **`mm_beam_orch_stochastic`** - best composite score overall (4.35, rank 1).

- **mm_beam_orch_stochastic_repair** - a near-identical practical variant (4.90, rank 2), but it leads on every transition metric: lowest p95 transition (1.789) and lowest max transition (2.625) in the entire study - while matching the best baselines on trajectory adherence (arc_rmse 0.176).
- Together, these two results validate our two core contributions to MM beam - stochastic pruning (more diverse outputs, no quality loss) and candidate orchestration (bottleneck- and arc-aware filtering that steers the beam toward coherent placements).

The repair pass is the key differentiator - it adds negligible runtime cost but buys the best worst-case transitions, making it the recommended production configuration. Skip repair only if the absolute lowest composite score or minimum runtime is the top priority.

4.4.2 Surprising finding: BIBS underperforms its own simpler variant

bibs_current ranks 16th overall (composite 13.55) - well below the plain mm_beam baseline (5th, 6.55). Its transition costs are notably higher (avg_trans 1.19 vs. 0.92), despite similar arc RMSE (0.184 vs. 0.176).

BIBS's defining feature recursive midpoint anchoring is also its weakness. Anchors are placed based on arc and bottleneck scores alone, without fully weighing the transition cost immediately before and after each anchor. Since every recursion level places an anchor independently, this can create high-cost "stitching points" at each midpoint.

MM beam avoids this because it scores every candidate on both arc *and* transition cost at every step - transitions are never an afterthought.

4.4.3 The repair pass is the most effective single intervention

Across all base algorithms, adding repair consistently cuts p95 transition by ~5–10% and max transition by ~6–15%, with no meaningful change to arc_rmse.

- **BIBS:** average transition drops from 1.192 → 1.098; p95 from 2.890 → 2.284.
- **MM stochastic:** max transition drops from 2.805 → 2.626.

Repair is cheap (~0.13–0.17s added runtime) and always safe - it never worsens arc adherence beyond the guard threshold. It's the single highest-leverage change you can make to any method. The effect of repair on worst-case transition behavior is summarized in Fig. A3.

4.4.4 The trajectory vs. transition tension

The core tension in the problem shows up clearly at the extremes:

- **arc_assignment** - best arc_rmse (0.094), but terrible transitions (avg_trans 2.187, global coherence 0.086).
- **transition_greedy** - best average transition (0.894), but weak arc-following (arc_rmse 0.199).
- **Our best method** - 0.176 arc_rmse and 0.913 avg_trans: a strong balance of both.

This is the core problem statement in action, a useful party playlist can't optimize for arc *or* transitions alone; it has to do both.

4.4.5 Stochastic mode: diversity at comparable quality

Stochastic variants exist because a practical system should generate different valid playlists from the same pool, not one fixed order every time. Instead of keeping only the top-scoring beams, stochastic pruning samples from strong candidates - preserving quality while allowing run-to-run variation.

MM Beam Stochastic performs almost identically to deterministic MM Beam on the main metrics - diversity with no meaningful quality loss. The repair-enhanced stochastic version even slightly improves coherence and average transition, making it one of the most practical methods overall.

4.4.6 Results are stable across N sizes and trajectory shapes

Relative method ranking is consistent regardless of playlist length; larger N slightly improves absolute transition quality (more candidates → better local choices). Across trajectory shapes, double_peak is easiest (best arc_rmse for most methods) while inverted_parabola is hardest for arc_assignment specifically (0.138 vs 0.075 for linear). The winning methods perform consistently across all five shapes and all three genre-diversity settings, so the approach generalizes across realistic use cases. The stability of the recommended method across playlist sizes is shown in Fig A4.

N	Best method	arc_rmse	avg_trans	global_coh
60	mm_beam_orch_stochastic	0.179	1.022	0.783
90	mm_beam_orch_stochastic	0.176	0.911	0.822
120	mm_beam_orch_stochastic	0.173	0.828	0.848

4.4.7 Negative results: alpha-boost, trajectory-first, and tempo-aware

Three variants didn't pan out:

- **Alpha-boost (rhythm weight $\alpha=2.0$):** Makes both bibs_alpha2 and mm_beam_alpha2 worse across the board. Over-weighting tempo differences trades away arc adherence and overall balance for tempo smoothness - the default equal weighting wins.
- **Trajectory-first endpoint selection (bibs_traj_first, rank 18):** No better than bibs_current (rank 16). The existing ev_proximity endpoint mode already handles start/end selection well, and this variant adds runtime overhead from an implementation bug.
- **Tempo-aware BIBS (bibs_tempo_aware, rank 17):** No metric improvement over bibs_current, but runs $\sim 1500\times$ slower ($\sim 4.5s$ vs. $\sim 0.003s$ per playlist) due to a Python-loop bottleneck in the penalty computation.

None of these three should be adopted as-is. The alpha-boost and trajectory-first ideas are simply outperformed by existing defaults. Tempo-aware BIBS is the one exception worth revisiting - the concept is sound, but it needs to be vectorized before it can be fairly evaluated; until then, it's excluded from the recommended comparison.

4.4.8 Bottleneck term analysis

The bottleneck mechanism was introduced to prevent restrictive tracks from being left until the end. Inside recursive BIBS its direct effect is limited on these relatively homogeneous pools, where few tracks are truly hard to place. However, bottleneck information remains useful inside the Candidate Orchestrator: combined with arc-cost filtering and transition-graph neighbors, it produces smaller, more relevant candidate pools.

4.5 Current System Architecture Summary

The final implemented architecture in `src/` consists of the following modules:

Module	File	Role
Feature Engineering	<code>src/feature_engineering.py</code>	Normalizes Spotify features and computes EV_score.
Trajectory Arcs	<code>src/arcs.py</code>	Generates target energy trajectories (linear, wave, double peak, etc.).
Cost Functions	<code>src/cost_functions.py</code>	Computes transition costs from rhythm, tonality, and texture.
Bottleneck Detection	<code>src/bottleneck_detector.py</code>	Identifies difficult tracks and starved playlist positions.
Transition Graph	<code>src/transition_graph.py</code>	Builds a sparse graph of low-cost transitions.
Candidate Orchestrator	<code>src/candidate_orchestrator.py</code>	Filters and ranks candidate tracks for search algorithms.
BIBS Core	<code>src/bibs.py</code>	Recursive bottleneck-guided bidirectional beam search.
Greedy Baseline	<code>src/greedy_baseline.py</code>	The greedy trajectory-transition baseline.
Other Baselines	<code>src/baselines/</code>	Random, Transition-Greedy, Hungarian, Flexer, Forward Beam, MM Beam.
Parallel Solver	<code>src/parallel_solver.py</code>	The parallel BIBS variant.
Sampling	<code>src/sampling.py</code>	Stochastic beam sampling.
Repair	<code>src/repair.py</code>	Locally fixes poor transitions while preserving the arc.
Evaluator	<code>src/evaluator.py</code>	Computes RMSE, transition costs, percentiles, coherence, DTW, diversity.
Experiment Runner	<code>src/experiments/runner.py</code>	Runs experiments and exports comparable results.

4.6 Limitations and Future Work

4.6.1 Current Limitations

The system uses pre-computed Spotify-style audio features rather than raw audio, so it does not perform beat matching, phrase alignment, crossfade detection, or waveform-level transition analysis; a transition that looks smooth in feature space may still sound imperfect. The transition-cost weights ($\alpha=1.0$, $\beta=0.4$, $\gamma=0.6$) are heuristic and were not learned from listener feedback, and different genres may need different weightings. BIBS preserves the global trajectory well but can create local transition artifacts at recursive anchor points; repair reduces but does not fully solve this. Finally, evaluation is computational rather than perceptual - the metrics are useful proxies, but full validation requires human listening tests.

4.6.2 Planned Future Work

1. **Listener study** - compare generated playlists against baselines to check whether computational metrics actually match human perception.
2. **User-controlled endpoints** - let the DJ pin the first and last track (partially implemented via quantile and trajectory_fit endpoint modes; needs UI integration and evaluation).
3. **Fix and re-evaluate bibs_tempo_aware** by vectorizing the penalty computation.
4. **Hybrid model** - BIBS for global structure + MM stochastic/repair for local fill, combining BIBS's trajectory discipline with MM beam's transition quality.
5. **Learn C_trans weights** and the arc-vs-transition trade-off from user preference data, potentially per genre.
6. **Proper multiprocessing implementation** of level-synchronous anchor placement (bibs_parallel) for large N.
7. **Online variant** - insert new tracks into an existing playlist without full recomputation.

4.7 Conclusions

This project developed a computational framework for ordering party playlists under two competing goals: following a planned energy trajectory and maintaining smooth transitions between songs, formulated as an exact-coverage optimization problem where every track appears exactly once.

Single-objective methods aren't sufficient: arc_assignment follows the energy arc best but produces poor transitions; transition_greedy does the opposite. BIBS contributes a valuable bottleneck-aware search framework and helps solve the leftovers problem, but its recursive midpoint anchoring hurts transition quality.

The strongest result is the MM-beam family - specifically **mm_beam_orch_stochastic** and its repair-enhanced counterpart - which combine bidirectional beam search, stochastic diversity, BIBS-style candidate orchestration, and guarded local repair. This family delivers the best composite results: strong transitions, competitive trajectory adherence, and run-to-run diversity for real-world use.

BIBS isn't a failure - it's the foundation. Its bottleneck detection and candidate orchestration are directly adopted by the winning methods, and the finding that replacing its recursion with a

committed bidirectional beam works better is itself a contribution: it shows exactly which parts of BIBS matter. our best methods significantly outperform all baselines on transition-smoothness while matching the best on trajectory adherence - and unlike every deterministic baseline, they produce diverse orderings per run, a practical requirement for DJ tooling.

Appendices

Appendix A - BIBS Pseudocode

```

Algorithm BIBS
Input: S (pool), L, s_start, s_end, A_EV(t), BIBSConfig
Output: P (ordered playlist)

Precompute:
  C_trajectory[s, t]  for all s in S, t in 1..L
  C_trans[u, v]       for all u, v in S
  BS_song[s], BS_loc[t] from C_trajectory
  TransitionGraph      (low-cost neighbor lists)

P[0] = s_start; P[L-1] = s_end
Available = S \ {s_start, s_end}

SOLVE_INTERVAL(0, L-1, Available):
  if (right_pos - left_pos - 1) <= base_case_size:
    fill_base_case(left_pos, right_pos, Available)
    return
  M = floor((left_pos + right_pos) / 2)
  fwd_beams = expand_beam(P[left_pos], positions[left_pos+1 .. M-1], forward)
  bwd_beams = expand_beam(P[right_pos], positions[right_pos-1 .. M+1], backward)
  anchor = select_anchor(M, fwd_beams, bwd_beams, Available)
  P[M] = anchor; remove anchor from Available
  SOLVE_INTERVAL(left_pos, M, Available)
  SOLVE_INTERVAL(M, right_pos, Available)

return P

```

Appendix B - Metric Definitions

Metric	What it measures	Direction
arc_rmse	RMSE between each track's EV score and the target trajectory value at its position. How well the playlist follows the intended energy shape.	Lower
average_transition_cost	Mean of $C_trans[i, i+1]$ across consecutive pairs. Overall smoothness.	Lower
p95 / max transition	95th percentile / maximum single-step transition cost. Worst-case jarring moments.	Lower
global_coherence	Mean correlation between EV score and a position-normalized smooth curve. How naturally energy flows.	Higher

Metric	What it measures	Direction
dtw_shape	Dynamic Time Warping distance between the actual EV sequence and the target shape. Tolerates slight timing shifts.	Lower
ratio_adherence_rmse	RMSE of each track's ratio position vs. the expected uniform ratio (Flexer-inspired acoustic interpolation quality).	Lower
method_time_s	Wall-clock time for the algorithm only (excludes shared precompute).	Lower
total_time_s	Wall-clock time including shared precompute (EV, C_trans, C_arc, bottleneck, graph).	Lower

EV score = weighted mean of 5 normalized audio features (energy, valence, danceability, normalized loudness, normalized tempo), all weights 1.0. $C_trans = \alpha \cdot \text{rhythm} + \beta \cdot \text{tonality} + \gamma \cdot \text{texture}$ where $\text{rhythm} = |\log_2(\text{bpm}_u/\text{bpm}_v)| + \text{meter penalty}$, $\text{tonality} = \text{Camelot circle-of-fifths distance}$, $\text{texture} = \text{L2 difference of texture features}$; defaults $\alpha=1.0$, $\beta=0.4$, $\gamma=0.6$.

Appendix C - Additional Evaluation Formulas

Anchor Alignment Cost (BIBS-specific). For any midpoint slot M selected by BIBS: $AC(x_M) = w_trajectory \cdot |EV_score(x_M) - A_EV(M)|^2 + w_trans \cdot [C_trans(x_M, x_M) + C_trans(x_M, x_M)]$. This verifies that meet-in-the-middle anchor placements are structurally sound and do not introduce acoustic drift where forward and backward beams meet.

Human Satisfaction Score (planned). $HSS = (1/N) \cdot \sum r_i$ over N participants, with r_i the rating of participant i on a 5-point Likert scale; we also planned to report the percentage of participants ranking each method first.

This evaluation was not carried out during the project due to time constraints; the formula is retained here for the planned future study.

Appendix D - Results

Table D.1 - Composite Ranking: All Methods

Rank	Method	Category	Composite Score
1	mm_beam_orch_stochastic	ours	4.35
2	mm_beam_orch_stochastic_repair	ours	4.90
3	mm_beam_stochastic	ours	5.10
4	mm_beam_stochastic_repair	ours	5.30
5	mm_beam	baseline	6.55
6	greedy_arc_trans	baseline	7.35
7	transition_greedy	baseline	7.60
8	mm_beam_alpha2	baseline	9.30
9	forward_beam	baseline	10.40
10	bibs_stoch_repair	ours	11.30
11	bibs_repair	ours	11.40
12	arc_assignment	baseline	11.60
13	bibs_parallel	ours	12.10
14	bibs_stochastic	ours	12.30

Rank	Method	Category	Composite Score
15	flexer_interp	baseline	12.80
16	bibs_current	ours	13.55
17	bibs_tempo_aware	ours	14.60
18	bibs_traj_first	ours	15.10
19	bibs_alpha2	ours	17.00
20	random	baseline	17.40

Table D.2 - Key Metric Snapshot: Overall Averages Across All 360 Playlists

Method	arc_rmse	avg_trans	global_coh	p95_trans	max_trans
mm_beam_orch_stochastic	0.176	0.920	0.818	1.827	2.804
mm_beam_orch_stochastic_repair	0.176	0.913	0.823	1.789	2.625
mm_beam_stochastic	0.176	0.920	0.818	1.828	2.805
mm_beam_stochastic_repair	0.176	0.913	0.823	1.789	2.626
mm_beam (baseline)	0.176	0.920	0.818	1.828	2.805
greedy_arc_trans (baseline)	0.184	0.909	0.814	1.853	3.293
transition_greedy (baseline)	0.199	0.894	0.820	1.813	3.267
mm_beam_alpha2 (baseline)	0.181	0.945	0.807	1.907	2.895
forward_beam (baseline)	0.185	0.934	0.807	1.903	3.129
bibs_stoch_repair	0.184	1.100	0.736	2.282	3.168
bibs_repair	0.184	1.098	0.736	2.284	3.167
arc_assignment (baseline)	0.094	2.187	0.086	3.522	4.090
bibs_parallel	0.171	1.553	0.489	2.972	3.869
bibs_stochastic	0.184	1.189	0.660	2.862	4.097
flexer_interp (baseline)	0.150	2.206	0.071	3.556	4.146
bibs_current	0.184	1.192	0.657	2.890	4.104
bibs_tempo_aware	0.183	1.193	0.656	2.916	4.063
bibs_traj_first	0.184	1.191	0.657	2.897	4.095
bibs_alpha2	0.186	1.240	0.623	3.119	4.086
random (baseline)	0.194	2.292	0.006	3.640	4.222

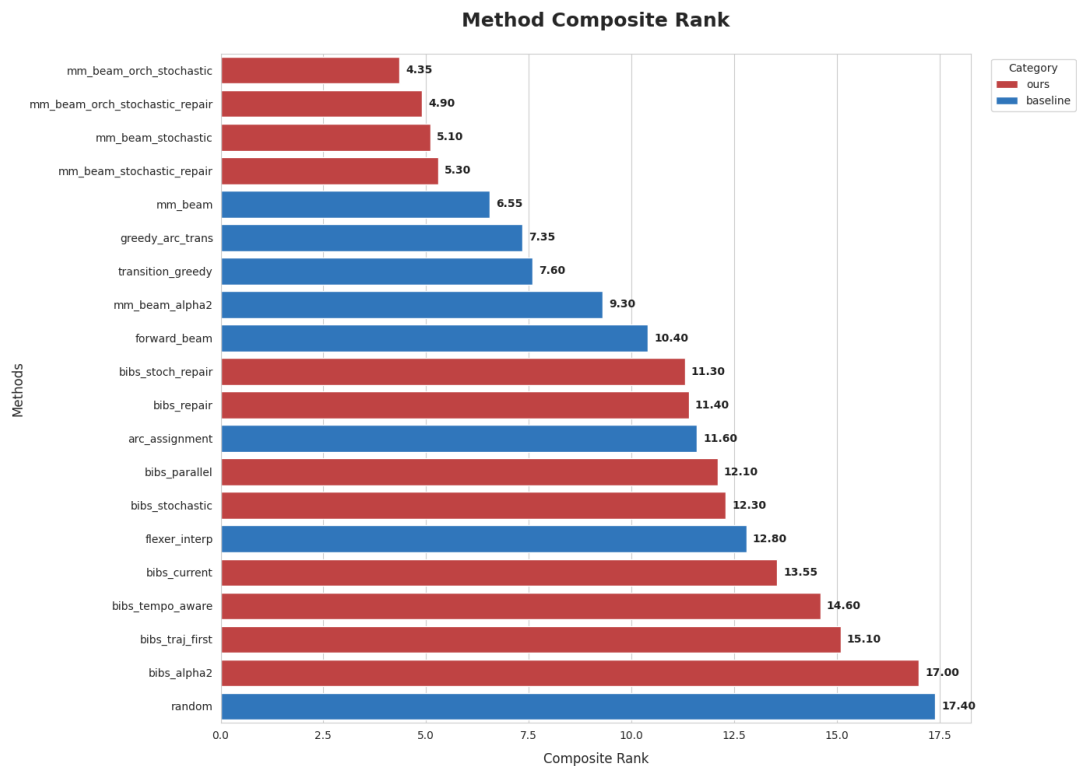


Fig. A1- Composite ranking across all methods; lower score is better. Our MM-beam stochastic variants occupy the top four ranks.

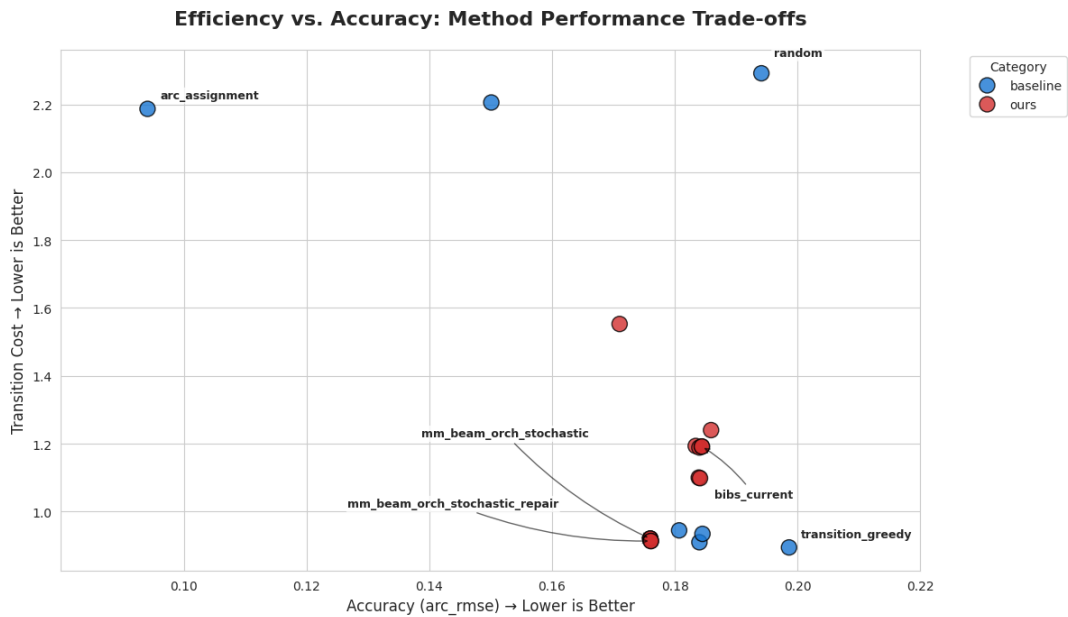


Fig. A2 -Trade-off between trajectory accuracy and transition smoothness. arc_assignment is best on arc RMSE, transition_greedy is best on transitions, while the recommended MM-beam variants balance both.

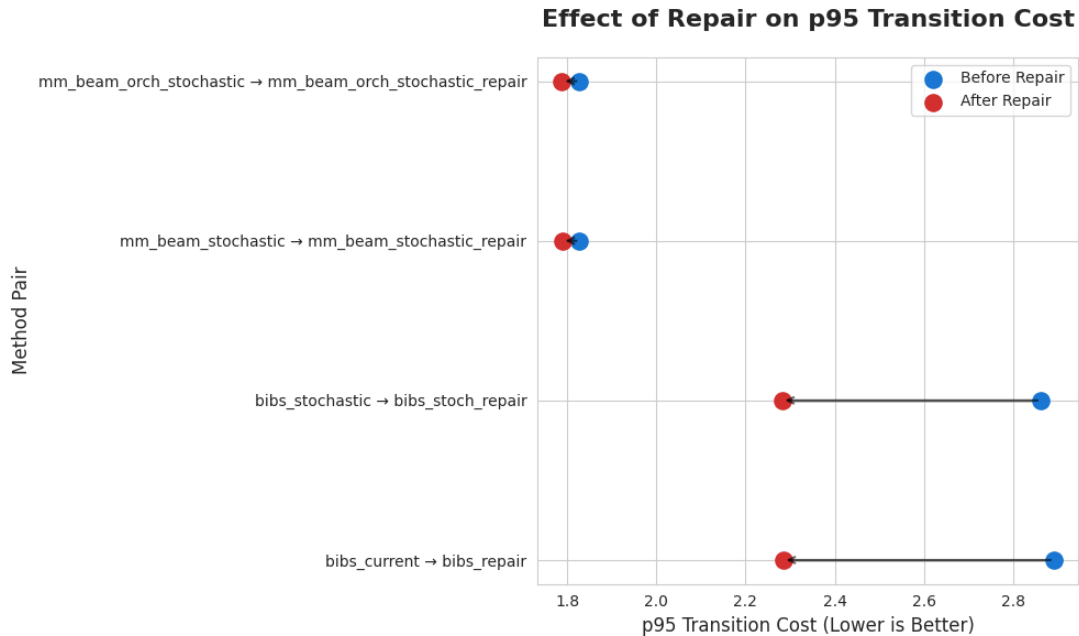


Fig. A3 - Effect of repair on p95 transition cost. Repair consistently reduces high-cost transition outliers, especially for BIBS-based methods.

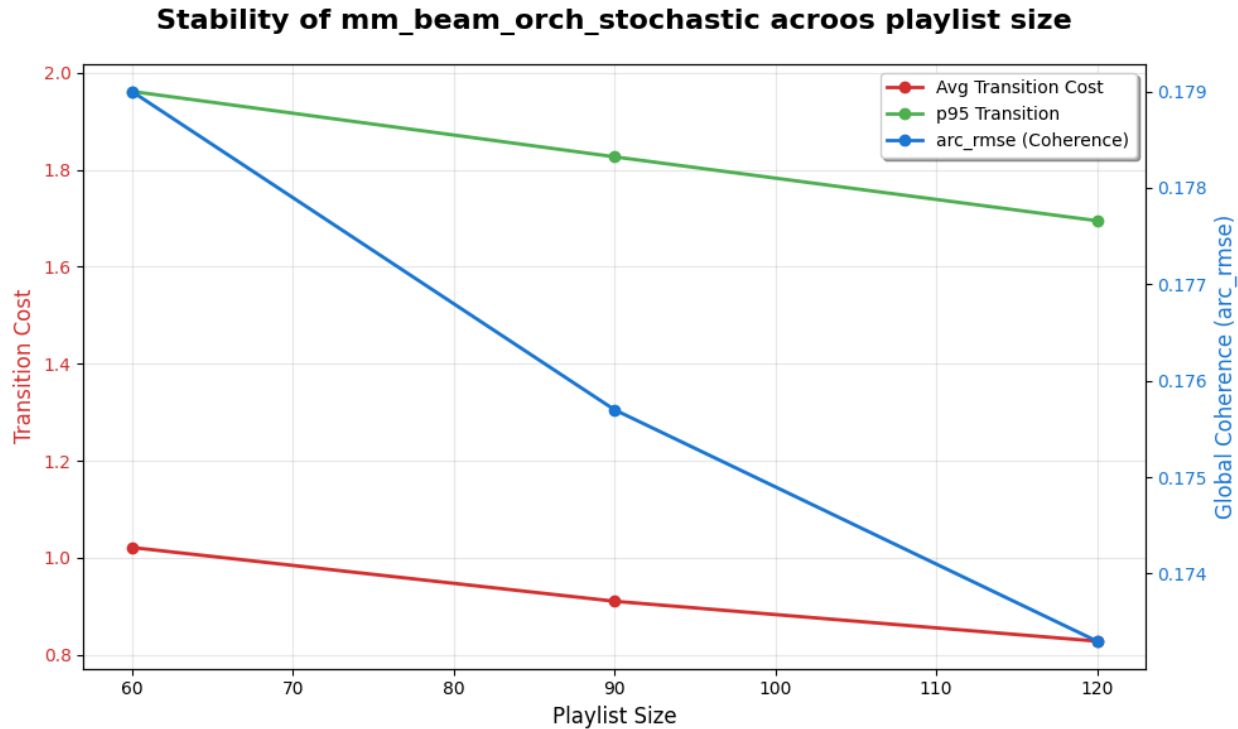


Fig. A4 - Stability of mm_beam_orch_stochastic across playlist sizes. Transition costs decrease as N grows, while arc RMSE remains stable.

Appendix E - Works Cited

- [1] F. Tomasi, J. Cauteruccio, S. Kanoria, K. Ciosek, M. Rinaldi and Z. Dai, “Automatic Music Playlist Generation via Simulation-based Reinforcement Learning,” 2023.
- [2] G. Bonnin and D. Jannach, “Automated Generation of Music Playlists: Survey and Experiments,” 2014.
- [3] A. Flexer, D. Schnitzer, M. Gasser and G. Widmer, “Playlist Generation Using Start and End Songs,” 2008.
- [4] R. M. Bittner, M. Gu, G. Hernandez, E. J. Humphrey, T. Jehan, P. H. McCurry and N. Montecchio, “Automatic Playlist Sequencing and Transitions,” 2017.
- [5] E. Liebman, M. Saar-Tsechansky and P. Stone, “DJ-MC: A Reinforcement-Learning Agent for Music Playlist Recommendation,” 2015.
- [6] M. Zehren, M. Alunno and P. Bientinesi, “Automatic Detection of Cue Points for the Emulation of DJ Mixing,” 2022.
- [7] L. V. Veire and T. D. Bie, “From Raw Audio to a Seamless Mix: Creating an Automated DJ System for Drum and Bass,” 2018.
- [8] H. Schweiger, E. Parada-Cabaleiro and M. Schedl, “The Impact of Playlist Characteristics on Coherence in User-Curated Music Playlists,” 2025.
- [9] R. C. Holte et al., “Bidirectional Search That Is Guaranteed to Be Optimal,” AAAI, 2016.